The Essential of the Tickets

Twenty-two discussion tickets across chapters 1 to 6, each answered in the rubric's shape: claim, mechanism, number, consequence; with the method for a ticket you have never seen

Advanced Data Management

2026-10-29

How to read this document

A discussion ticket is a colleague's claim or proposal, usually a wrong rule of thumb ("always", "never", "more is better"), a mystery ("why is it slow"), or a setting someone wants to change. It is graded on four things, in this order: a **claim** (agree or disagree, in one sentence), the **mechanism** (what the engine actually does, with the chapter's vocabulary), a **number** (one computation that makes the mechanism concrete), and the **consequence** (what happens if the claim is followed, and what to do instead or what to measure). Half a page is the budget; four short paragraphs with those four labels is the safest layout.

The tickets below cover both mocks (marked) and the other claims a chapter can generate. Each answer is written at the length the exam wants. The last section gives the method for a ticket not on this list: every chapter produces tickets from the same three or four mechanisms, so an unseen ticket is almost always a seen mechanism in new clothes. Numbers to carry: RAM 0.1 μs, SSD random read 100 μs, HDD 10 ms, sequential 500 MB/s, page 8 KB; `orders` 500,000 rows / 3,185 pages (157 per page), `customers` 10,000 / 73; cost weights sequential page 1, random page 4, row 0.01.

1 Chapter 1: What a DBMS Is For

Ticket: T1

"We do not need a database; CSV files and a Python script are faster"

Answer

Claim. For one user reading one file once, yes; for anything shared, updated and relied upon, no, and speed is the least of the reasons.
Mechanism. A DBMS provides five things files cannot: declarative

querying (you state what, the optimizer chooses how, and reuses indexes and statistics you never wrote), concurrency control (two clerks updating the same balance do not lose one update), recovery (a crash mid-write leaves either the old or the new state, never half a file), integrity (types, keys, foreign keys enforced on every write), and physical independence (the layout can change without touching any program).
Number. Finding one order among 500,000 by scanning a file reads 3,185 pages; a B^+-tree lookup reads 4. The script wins only until the first concurrent writer or the first crash.
Consequence. The script re-implements, badly, the parts of a DBMS it needs as soon as data is shared; the usual outcome is a corrupted file and no way to know which rows were lost.

Ticket: T2

"The query was fast last month and nobody changed the SQL, so the database is broken"

Answer

Claim. The SQL is not the plan; the plan is chosen from statistics and sizes, and those changed.
Mechanism. The query processor parses, binds, rewrites and then *optimizes*: it costs candidate physical plans from the catalog's statistics (row counts, distinct values, histograms) and picks the cheapest. If the table grew, the distribution shifted, or `ANALYZE` has not run since, the cost estimates moved and a different plan won, or the old plan is still chosen on numbers that are no longer true.
Number. A nested loop chosen for a believed 2,000-row outer costs about $2{,}000 \times 53 \approx 106{,}000$ page reads on paper; at 20,000 real rows it is 1.06 million, ten times the estimate, while a hash join would still read the inner once.
Consequence. Run `EXPLAIN (ANALYZE, BUFFERS)`, compare estimated and actual rows from the leaves up, run `ANALYZE` on the tables that grew, and only then talk about indexes or settings. The plan, not the text, is what runs.

2 Chapter 2: Pages, Files and Storage Models

Ticket: T3

"Why count pages and not CPU instructions?" (mock 1, question 2.2)

Answer

Claim. Because one disk page read costs as much as tens of thousands of instructions, so the page count predicts the running time and the instruction count does not.

Mechanism. Data moves between disk and memory in fixed 8 KB pages; a query pays per page touched, whether it needs one byte or all of them. RAM access is 0.1 μs, an SSD page 100 μs, an HDD page 10 ms: a gap of 10^3 to 10^5. The cost model keeps CPU terms only to break ties, which is exactly what PostgreSQL's weights say: sequential page 1.0, random page 4.0, one row 0.01, one operator 0.0025.
Number. Scanning `orders` costs 3,185 page units of I/O and $500{,}000 \times 0.01 = 5{,}000$ of CPU, comparable on paper; in time, 3,185 HDD random reads are 32 s while 500,000 row comparisons are milliseconds. Sequential versus random matters too: 25 MB streams in 50 ms at 500 MB/s.
Consequence. Every formula in chapters 2 to 6 counts pages (scan B, sort $2B\times$ passes, hash join $B_R + B_S$), and every design choice (indexes, clustering, buffer pool, ring buffers) exists to cut page reads or make them sequential. The exception to name: an in-memory table or a CPU-heavy filter, where the CPU terms decide.

Ticket: T4

"Column stores are better; migrate the order system to one"

Answer

Claim. Better for analytics, worse for the order system; the workload decides the layout.
Mechanism. A row store keeps each row's fields together in one page: one page read returns a whole order, and an insert writes one page. A column store keeps each column in its own file: a query that sums one column of twenty reads one twentieth of the bytes, and same-typed values compress well; but reading one whole row touches twenty files, and inserting one row writes twenty.
Number. Summing `amount` over 500,000 orders: row store 3,185 pages; column store about $3{,}185/20 \approx 160$ pages before compression, fewer after. Fetching one order: row store 1 page; column store about 20.
Consequence. OLTP (many small reads and writes of whole rows) stays on rows; OLAP (few queries scanning millions of rows, few columns) goes to columns, often as a nightly copy. Migrating the order system makes every insert twenty times more expensive.

Ticket: T5

"We deleted half of the rows, so the table file is now half the size" (mock v2, A2)

Answer

Claim. No: the rows are dead but their pages are still there; only a rewrite shrinks the file.
Mechanism. `DELETE` marks tuples dead inside their pages. A plain `VACUUM` reclaims that space *within* each page and records it in the free-

space map for future inserts; it does not move live rows between pages, so it can only truncate trailing pages that became entirely empty. With random deletes every page keeps about half its rows and none empties. `VACUUM FULL` or `CLUSTER` rewrite the table compactly, with an exclusive lock.

Number. `orders`: 3,185 pages before, 3,185 after `DELETE` plus `VACUUM` (each half full), about 250,000/157 ≈ 1,592 after `VACUUM FULL`. A sequential scan still costs 3,185 until then.

Consequence. Scans do not get faster after the delete; inserts do get cheaper because the free space is reused. Rewrite only if the space matters and the lock is affordable.

3 Chapter 3: Indexes

Ticket: T6

"Index every column, indexes make everything faster" (mock 1, question 2.3)

Answer

Claim. An index speeds some reads and slows every write; most "just in case" indexes are never used and cost in full.

Mechanism. An index helps only a predicate that keeps a small fraction of the table: an index scan costs about one random page read (cost 4) per matching row, a sequential scan costs one sequential read per page, so the index wins below $s < 1/(4 \times \text{rows per page})$, well under 1% before bitmap scans soften it. Every index also costs space (16 bytes per row and more) and write amplification: each insert writes the heap page plus one leaf page per index, each update of an indexed column deletes and reinserts an entry.

Number. On `orders` the index wins below $3{,}185/(4 \times 500{,}000) = 0.16\%$, about 800 rows. A table with six indexes does about seven page writes per inserted row instead of one.

Consequence. Start from the queries actually run, index the columns they filter, join and sort on with the selective column first, create, measure with `EXPLAIN (ANALYZE, BUFFERS)`, and keep the index only if the plan changed and got faster.

Ticket: T7

"I created the index and the planner still does a Seq Scan, so the planner is broken"

Answer

Claim. The planner is almost certainly right; check selectivity, correlation and the predicate's shape before blaming it.

Mechanism. Three reasons an index is ignored. (1) Selectivity: the predicate keeps too many rows (above a few percent), so random heap fetches cost more than one sequential pass. (2) Correlation: even at 8% the matching rows are spread over every page when `pg_stats` correlation is near 0, so a bitmap scan touches all 3,185 pages anyway; near 1 the rows sit in consecutive pages and the index wins. (3) Shape: a function on the column (`EXTRACT(year FROM d)`, `cust_id + 0`) or a non-leading column of a composite index cannot use the tree.

Number. Mock v2 C4: `order_date >= '2025-12-01'` keeps 8.7%, 43,388 rows, correlation -0.0015: about 13 matching rows per page, so the index would read $3{,}185 \times 4 = 12{,}740$ against the scan's 3,185.

Consequence. Either narrow the predicate, cluster the table on the column (or use BRIN for a correlated date), rewrite the predicate to be index-friendly, or accept the scan. "Seq Scan" on a query that returns half the table is correct behaviour.

Ticket: T8

"Put the date first in the composite index, dates are what we filter on"

Answer

Claim. Lead with the column tested by equality and most selective; a range column goes last.

Mechanism. A B^+-tree on (a, b) sorts by a then b. A predicate can descend on b only within one value of a (leftmost-prefix rule). With `(cust_id, order_date)`, `cust_id = 7 AND order_date > d` descends to customer 7's run and walks the date range inside it; with `(order_date, cust_id)` the same query descends on the date range and must filter `cust_id` across every customer in that range. A query on the second column alone cannot use the index at all.

Number. `cust_id = 7` keeps 50 rows; the date range keeps 43,388. Index on `(cust_id, order_date)`: 3 index pages plus at most 50 heap fetches. Index on `(order_date, cust_id)`: 43,388 entries walked to find 4.

Consequence. Mock v2 A3: the best user of `(cust_id, order_date)` is `cust_id = 7 AND order_date > ...`; a date-only query needs its own index, and `cust_id + 0 = 7` uses none.

Ticket: T9

"The lookup API must answer in 1 ms; buy more RAM" (mock v2, B2)

Answer

Claim. What is bought is the disk, not the RAM: the latency is a handful of random page reads, and only the storage decides what each costs.
Mechanism. A lookup descends the B$^+$-tree, one page per level, then one heap page. Levels $= \lceil \log_F N \rceil$ with F entries per page. The root and the upper levels are hot and stay in shared buffers; the leaf and the heap page are the misses, and a miss costs 100 μs on SSD, 10 ms on HDD.
Number. 20,000,000 rows, 250 per page: $\lceil \log_{250} 2 \times 10^7 \rceil = 4$ levels, 5 reads. Cold: 0.5 ms SSD, 50 ms HDD. Root and level 2 cached: 2×0.1 μs $+3 \times 100$ μs ≈ 0.3 ms SSD, 30 ms HDD.
Consequence. On HDD no amount of RAM short of caching the whole 630 MB index and heap meets 1 ms, and the first cold miss still costs 10 ms; on SSD the target is met with the default cache. Keep the SSD; add RAM only if the measured hit ratio shows leaves missing.

4 Chapter 4: Sorting and Joining

Ticket: T10

"Hash join is always the best; disable nested loop and merge join server-wide" (mock v2 D2, mock 1 question 2.4)

Answer

Claim. Each algorithm is the best choice exactly where another is a disaster; disabling two of them server-wide forces the disaster on every query of those shapes.
Mechanism. Hash join: build a table on the smaller input, probe with the larger; $B_R + B_S$ when the build fits `work_mem`, $3(B_R + B_S)$ when it spills (Grace, `Batches` > 1); equality only. Index nested loop: for each outer row, descend an index on the inner; $B_R + |R| \times (h + \text{matches})$; unbeatable when the outer is tiny and the inner is indexed, catastrophic when the outer is large. Sort-merge: sort both then one linear merge; wins when the inputs are already sorted (index scans) or sorted output is wanted, and it handles inequalities, which hashing cannot.
Number. Mock v2 B3 at $M = 100$: block nested loop 3,280 against hash 3,280, a tie. The lab's `WHERE c.id < 10`: nine customers, index nested loop 0.26 ms against a hash join that must still read all 3,185 pages of `orders`. `WHERE r.start < s.ts`: no hash join exists.
Consequence. With the switches off, point lookups joined to a big table read the whole big table every time, range joins fall back to a block nested loop over everything, and queries wanting sorted output sort after a hash join instead of merging. `enable_*` are session-level experiment tools, followed by `RESET`; never a server setting.

Ticket: T11

"Doubling work_mem will halve the sort time" (mock v2, B1)

Answer

Claim. Memory buys passes, not percentages: doubling it changes the time only when it removes a pass, and then by a third or a half, not by whatever the ratio was.
Mechanism. External merge sort writes $\lceil B/M \rceil$ sorted runs, then merges $M-1$ at a time; passes $= 1 + \lceil \log_{M-1} \text{runs} \rceil$, and each pass reads and writes every page, $2B$. The I/O is $2B\times$ passes: a step function of M.
Number. $B = 1{,}200$: $M = 25$ gives 48 runs, fan-in 24, two merge passes, three passes, 7,200 I/O; $M = 50$ gives 24 runs, fan-in 49, one merge pass, two passes, 4,800 I/O, a third less. A single pass needs $M \geq 1{,}200$ pages (9.6 MB); one merge pass needs $M = 36$.
Consequence. The DBA is wrong by arithmetic but right by luck here (3 passes to 2). The useful reply: name the threshold, set `work_mem` just above it for this report with `SET LOCAL`, and check the plan says `quicksort` instead of `external merge`.

Ticket: T12

"Set work_mem = 1GB for everyone, sorts will never spill again"

Answer

Claim. `work_mem` is per sort or hash operation per connection, not per server; 1 GB for everyone is a promise the machine cannot keep.
Mechanism. Every Sort node, Hash node and HashAggregate in every running query may allocate up to `work_mem`, outside `shared_buffers`. A query with three such nodes and a hundred concurrent connections multiplies the setting by three hundred. When RAM runs out the OS swaps, which is worse than the spill the setting was meant to avoid.
Number. 100 connections, 2 operators each, 1 GB: 200 GB on a 64 GB server. The same with 64 MB: 12.8 GB, fine next to a 16 GB pool.
Consequence. Keep the server default modest (a few to a few tens of MB), raise it per session for the known heavy report (`SET LOCAL work_mem = '256MB'` inside its transaction), and size it from the plan: mock v2 C5's 519 kB hash needed 8 batches at 64 kB and one batch at 1 MB.

Ticket: T13

"Batches: 8 in the plan means the hash table is corrupted"

Answer

Claim. It means the build side did not fit in `work_mem`, so the join ran as a Grace hash join in eight partitions; nothing is broken, it is just three

times the I/O.
Mechanism. When the build exceeds memory, both inputs are partitioned to disk by a hash of the join key into k batches, then each pair of partitions is joined in memory. Each input is written once and read once more: about $3(B_R + B_S)$ instead of $B_R + B_S$. The plan shows it as `Batches: 8`, `temp written` under the Hash node and `temp read/written` at the join.
Number. Mock v2 run 3: `work_mem = 64kB`, build of 519 kB, 8 batches, `temp written=28` on the build and 183 at the join; execution 71.5 ms against 62.3 ms with one batch.
Consequence. Raise `work_mem` for that query above the build size (1 MB here, 4 MB for margin) and the line reads `Batches: 1` with no temp counters.

5 Chapter 5: The Optimizer

Ticket: T14

"Why is cardinality estimation the hardest part of optimization?" (mock 1, question 2.5)

Answer

Claim. Because every other decision is arithmetic on the row counts, and the row counts are the one thing the optimizer must guess about data it has not read.
Mechanism. Rewrites are mechanical and always safe. Costing is exact: given sizes, the chapter 4 formulas price every plan. But the sizes come from statistics summarised at the last `ANALYZE`: 1/n_distinct for equality, a histogram for ranges, the product for AND, $|R||S|/\max(d_R, d_S)$ for a join. Each rule has a failure: stale statistics (the table grew), correlated columns (the product under-estimates), skew (a value missing from the MCV list). And an error at a leaf is multiplied at every join above it, because each operator's estimate is computed from its children's.
Number. Practice B2: independence gives 20 customers where 3,000 are real, q-error 150; the join estimate inherits the same 150; the index nested loop chosen on 20 rows reads 159,001 pages against the hash join's 3,186.
Consequence. Access path, join algorithm and join order all flip on row counts, so a wrong leaf turns the cheapest plan into the worst. Diagnose from the leaves up with estimated versus actual, and fix the statistics before anything else.

Ticket: T15

"The planner chose a terrible plan; we need optimizer hints like other databases"

Answer

Claim. A terrible plan is almost always a wrong estimate; fix what the planner knows before overriding what it decides.
Mechanism. The lever order PostgreSQL expects: (1) statistics, `ANALYZE`, then `CREATE STATISTICS (dependencies)` on correlated filter columns, then a larger statistics target so skewed values enter the MCV list; (2) cost parameters that describe the hardware, `random_page_cost` near 1.1 on SSD, `effective_cache_size`; (3) memory, `work_mem` so the right hash or sort does not spill; (4) `SET enable_nestloop = off` in one session to see the second-best plan, then `RESET`; (5) `pg_hint_plan` only for a pattern the statistics cannot describe.
Number. Practice C: an estimate of 4 rows against 45 for one customer's December orders is a per-customer pattern no table statistic captures; it is harmless there (2-page build side) and the one case a hint might serve.
Consequence. A hint freezes one plan for one data size; next year the hinted nested loop that was right at 2,000 rows is wrong at 20,000 and nobody remembers it exists, while the planner with correct statistics would have changed its mind on its own.

Ticket: T16

"We ran ANALYZE and the estimate is still off by 100×, so statistics are useless"

Answer

Claim. `ANALYZE` fixes stale statistics; the remaining 100× is one of the other two failures, each with its own fix.
Mechanism. Per-column statistics are correct after `ANALYZE`; the error is in how they are combined. Correlated columns: the planner multiplies sel(country) × sel(city) although city implies country, so the AND is under-estimated by the factor sel(country); `CREATE STATISTICS (dependencies) ON country, city` then `ANALYZE` tells it. Skew: a very common value absent from the MCV list is estimated at 1/n_distinct; raising the column's statistics target lengthens the list.
Number. Casablanca: $0.40 \times 1/200 \times 10{,}000 = 20$ estimated, 3,000 real. Dependencies alone bring it to 50 (the city's uniform share); the MCV entry brings it to 3,000.
Consequence. Always ask which of the three failures it is: a wrong base count (stale), a wrong product (correlation) or a wrong single-value fraction (skew). Each has one command.

Ticket: T17

"Our disks are SSDs; set random_page_cost = 1 everywhere"

Answer

Claim. Lower it on SSD, but to about 1.1 to 1.5 and per tablespace, after measuring; 1.0 flips plans that should not flip.
Mechanism. The parameter is the price of a page fetched through an index relative to a sequential page at 1.0. It sets the access-path threshold $s < B/(\text{random_page_cost} \times N)$ and the relative price of index nested loops. A random SSD read is still one 8 KB request at 100 μs against 16 KB per 100 μs streamed, and sequential scans also get read-ahead and ring buffers.
Number. On `orders`, threshold 0.16% at 4.0, 0.64% at 1.0: about 800 rows against 3,200. Every under-estimate is then amplified: a loop chosen on a believed 20 rows that are 3,000 is 150× the I/O.
Consequence. Queries in the 0.16–0.64% band move to index scans; joins with a small-looking outer move to nested loops. Measure with the slow-query log and `EXPLAIN (ANALYZE, BUFFERS)` before and after a per-session `SET`, compare times not costs, keep 4.0 on spinning disks.

6 Chapter 6: The Buffer Pool

Ticket: T18

"Why does the DBMS run its own buffer pool instead of the OS page cache?" (mock 1, question 2.1)

Answer

Claim. Because the DBMS knows three things the OS cannot, and one of them is required for durability.
Mechanism. (1) The access pattern: a sequential scan touches each page once, a nested-loop inner is re-read per outer row, an index root is needed by every lookup; a generic LRU treats all as "recently used", the DBMS gives scans a 256 KB ring, pins pages in use and prefetches. (2) Write ordering: the log record must reach disk before the data page it describes and at commit; the OS flushes dirty file pages whenever it likes, so recovery would find pages with no log to undo them. (3) Memory: one copy in the OS and one in the DBMS is double buffering; the DBMS sizes its pool deliberately.
Number. Under OS LRU a 10,000,000-page nightly scan through 1,000 frames evicts all 200 hot pages; through a 32-frame ring it evicts none. A commit that forced 300 data pages instead of one log append would take 3 s on HDD instead of 2 ms.
Consequence. Without its own pool the DBMS could not guarantee write-ahead logging, so no crash recovery; scans would flood the cache every night; and RAM would hold two copies of every page.

Ticket: T19

"64 GB server, set shared_buffers = 52GB, more cache is always better" (mock v2, D1)

Answer

Claim. Disagree: PostgreSQL relies on memory outside the pool, and 80% starves all of it.
Mechanism. The OS page cache serves WAL writes, temporary files and the pool's "read" pages (a miss in the pool is cheap only because the OS still has the page). `work_mem` is allocated per operator per connection outside the pool. The checkpointer must write every dirty page in the pool each interval, so a larger pool means a larger burst.
Number. 100 connections at 64 MB of `work_mem` need 6.4 GB; with 12 GB left for the OS and that, pool misses become disk reads at 10 ms instead of OS-cache hits at microseconds. A 52 GB pool can accumulate tens of GB of dirty pages per checkpoint.
Consequence. Swapping, external sorts and multi-batch hashes, checkpoint stalls every few minutes. Start at 25% (16 GB), measure the hit ratio in `pg_statio_user_tables` and the usage counts in `pg_buffercache`, raise only while the hit ratio responds, stop near 40%.

Ticket: T20

"COMMIT takes 2 ms but changed 300 pages; how can the data be safe?" (mock v2, D3)

Answer

Claim. At commit PostgreSQL writes only the log, not the pages; the log is what makes the data safe.
Mechanism. Steal + no-force: dirty pages may be written whenever the pool likes and need not be written at commit. Two rules keep that safe: a page's log records reach disk before the page does, and the log up to the commit record is flushed (`fsync`) before commit returns. Commit is therefore one sequential append of a few KB; the pages follow via the background writer or the next checkpoint.
Number. 300×8 KB $= 2.4$ MB at random positions: a forced commit would wait for 300 random writes, 3 s on HDD, 30 ms on SSD; the log append is 2 ms.
Consequence. After a power cut, recovery replays the log from the last checkpoint: every change of a transaction with a commit record is redone, so the committed transaction loses nothing; a transaction without a commit record is aborted, and its tuple versions are simply invisible under MVCC (undone, in the general case), so it leaves no trace.

Ticket: T21

"Our cache hit ratio is 99%, so the disk is not the problem"

Answer

Claim. At 99% the disk *is* the problem; think in miss ratios, not hit ratios.
Mechanism. EAT $= h\,T_{\text{hit}} + (1-h)\,T_{\text{miss}}$. With a miss 10^5 times a hit, the second term dominates until $1-h$ is tiny: one miss in a hundred makes the average access a thousand times slower than a hit.
Number. HDD: $0.99 \times 0.1 + 0.01 \times 10{,}000 \approx 100$ μs per access, against 0.1 μs for a hit; EAT ≤ 1 μs needs $h \geq 0.99991$. SSD at 98%: 2.1 μs, 21 times a hit.
Consequence. Going from 99% to 99.9% removes nine of every ten disk reads and divides the average access by ten. Measure the miss count, not the percentage, and ask which pages miss (leaves, heap pages of a scan, a flooding report).

Ticket: T22

"The nightly report makes every morning slow; it is a different query, it cannot be related" (mock 1, question 2.6)

Answer

Claim. It is related: a one-pass scan larger than the pool evicts the hot working set, and the morning pays to reload it.
Mechanism. Under LRU every page of a sequential scan arrives as "most recently used" and pushes out pages the daytime workload reuses constantly. After more scan pages than frames, nothing hot is left. PostgreSQL defends with a ring buffer: a scan of a table larger than `shared_buffers`/4 cycles through 256 KB (32 frames) and never touches the rest of the pool. A scan that fits under the threshold goes through the main pool, and so does any index-driven access.
Number. Pool 1,000 frames, hot set 200 pages, scan of 10^7 pages: 0 hot pages survive under LRU, 200 misses the next morning; with a 32-frame ring all 200 survive. At 128 MB the threshold is 4,096 pages: `orders` (3,185) is not ringed, a 63,695-page archive is.
Consequence. Check whether the report's tables exceed the threshold (if not, it floods through the main pool), run it through a ring or off the main pool, or pre-warm the hot set afterwards; and `Buffers: read` in the first morning queries is the fingerprint.

7 A Ticket You Have Never Seen

Technique: The method

1. **Name the mechanism.** Every ticket is one of about fifteen: declarative versus plan (T2), pages not bytes (T3), row versus column (T4), dead tuples (T5), selectivity threshold (T6, T7), leftmost prefix (T8), tree levels and latency (T9), which join wins (T10), sort passes (T11),

per-operator memory (T12, T13), estimation and its three failures (T14–T16), cost parameters (T17), own pool and write ordering (T18), memory outside the pool (T19), WAL at commit (T20), EAT (T21), flooding and rings (T22). Find which one the claim violates.

2. **Decide, in one sentence.** "Disagree, because the DBMS ..." or "Right, but only when ...". A ticket that is partly right (T11, T17) earns more when you say exactly where.
3. **Write the mechanism in the chapter's words**, three to five sentences, defining each term you use (steal, ring, q-error, Grace).
4. **Put one number on it.** Use the lab's tables and the orders of magnitude; a formula with values substituted and a unit. One number, computed correctly, beats three guessed.
5. **State the consequence and the action**: what breaks if the claim is followed, what to measure (`EXPLAIN (ANALYZE, BUFFERS)`, `pg_stats`, `pg_buffercache`), what to set and at which scope (session, not server).

Warning / Gotcha: Four ways to lose the points

- Agreeing or disagreeing without a mechanism: the letter is worth little, the "because" is the grade.
- A mechanism from the wrong chapter: `work_mem` is chapter 4 memory, not the buffer pool; `read` in `Buffers` is not the disk.
- No number, or a number without a formula. "It would be much slower" earns nothing; "300×10 ms $= 3$ s against 2 ms" earns the point.
- A server-wide fix for a one-query problem: `enable_*`, `work_mem`, cost parameters are set per session or per query first.